194.048 Data Intensive Computing (VU 2,0) 2026S

Assignment 1

Group 20:

Doan Chau Tran (11736088)
Kashir Hasnain (12350030)
Sajjad Ahmad (12510832)
Faisal zada (12454530)
Adil Shinwari (12451264)

Date:

24/04/2026

Table of Contents

Introduction 3
Problem Overview 3
Text Preprocessing Tasks 3
1. Tokenization 3
2. Case Folding 3
3. Stopword Filtering 4
4. Removal of One-Character Tokens 4
5. Expected Output 4
Dataset: 5
Chi-Square Mathematical Definition 5
Methodology and Approach 6
Pipeline Architecture: 6
Key Value Design 6
1. Key Value Design for the Job 1 - Counting Step 6
2. Key Value Design for the Job 2 — Chi-Square Computing & Ranking 7
Job 1 - Counting Step 7
Mapper 7
Combiner 7
Reducer 8
Job 2 - Chi-Square Computing and Ranking 8
Mapper (Pass through) 8
Reducer (Chi-Square Computation) 8
Top-75 Selection and Merged Dictionary 9
Effeciency Considerations 9
Script running 10
Hadoop Configuration (pipeline_mrjob.conf) 11
Conclusions 11

Introduction

Here we use Python library (mrjob) on a 12 node Hadoop Cluster which processes a large scale product review and computes chi-square(χ^2) term ranking for text classification purposes. Before implementation we have to do some preprocessing tasks such as Tokenization, Case Folding, Stopword filtering, Calculate chi-square values, Order the terms and Merge the List.
In this work, we are applying MapReduce programming techniques like Input Spliting,Mapping, Shuffle and Sorting on the Dataset provided by the Amazon Review published in 2014. The dataset contains product reviews of 142.8 million reviews spanning May 1996 - July 2014. This dataset includes reviews (ratings, text, helpfulness votes), product metadata (descriptions, category information, price, brand, and image features), and links (also viewed/also bought graphs).

Problem Overview

This section describes the dataset used in the assignment, the preprocessing steps applied to prepare the text, the chi-square statistical method used to rank terms, and the required output format.In this assignment we have to do few text preprocessing tasks which includes:

Text Preprocessing Tasks

1. **Tokenization**

   Each review text shall be segmented into unigram tokens (single terms).
   Token boundaries are defined by the following delimiters:
   * Whitespaces
   * Tab characters
   * Numerical digits (0–9)
   * Special characters: () [] {} . ! ? , ; : + = - _ " ' ` ~ # @ & * % € $ § \ /

   Code:

```
def tokenize(text: str, stopwords: set[str], min_len: int) -> list[str]:
    """Extract alphabetic tokens, skipping short words and stopwords."""
    tokens = TOKEN_RE.findall(text)
    return [tok for tok in tokens if tok not in stopwords and len(tok) >= min_len]
```

2. **Case Folding**

   All alphabetic characters shall be converted to lowercase in order to avoid treating semantically identical terms with different capitalization as separate tokens.

Example:
"Great Product" → "great product"

3. **Stopword Filtering**

   All tokens contained in the provided stop word list (`stopwords.txt`) shall be removed. This step reduces the influence of high-frequency function words that do not contribute meaningful discriminatory information for category classification.

   Examples include:
   the, is, and, a, of, to
   Code:

```
def load_stopwords(path: Path) -> set[str]:
    """Load and return a set of lowercase stopwords."""
    with path.open("r", encoding="utf-8") as f:
        return {line.strip().lower() for line in f if line.strip()}
```

4. **Removal of One-Character Tokens**

   All tokens consisting of only one character shall be discarded, as they generally provide limited semantic relevance.

   Examples:
   a, i, x

5. **Expected Output**

   The preprocessing stage shall output a cleaned sequence of unigram terms for each review text, ready for subsequent MapReduce processing and chi-square computation.
   Example
   Input review text:
   "This is a GREAT product!!! 100% recommended."

   Output tokens:
   great, product, recommended

Dataset:

The whole dataset is available on HDFS and available here. Each line of the JSON object containing:

- **reviewerID** - string - the ID of the author of the review.
- **asin** - string - unique product identifier.
- **reviewerName** - string - name of the reviewer.
- **helpful** - array of two integers [a,b] - helpfulness rating of the review: a out of b customers found the review helpful.
- **reviewText** - string - the content of the review; this is the text to be processed.
- **overall** - float - rating given to product asin by reviewer reviewerID.
- **summary** - string - the title of the review.
- **unixReviewTime** - integer - timestamp of when review was created in UNIX format.
- **reviewTime** - string - date when review was created in human readable format.
- **category** - string - the category that the product belongs to.

Chi-Square Mathematical Definition

The Chi-square test measures the statistical independence between a **term t** and **catagory c**.

	Category c	Not Category c	Total
Term t	**A = N□^{c}**	B = N□$^{\overline{c}}$	A + B
No Term	**C**	D	C + D
Total	**N^{c}**	N$^{\overline{c}}$	**N**

Chi-Square is Computed at:

$$\chi^2(t, c) = N \cdot (A \cdot D - B \cdot C)^2 \;/\; ((A+B) \cdot (C+D) \cdot (A+C) \cdot (B+D))$$

A = N□^{c} — number of documents in category c that contain term t.

B = N^{c} − A — number of documents in category c that do NOT contain term t.

C = N□ − A — number of documents NOT in category c that contain term t.

D = N − A − B − C — number of documents NOT in category c and NOT containing term t.

Methodology and Approach

Pipeline Architecture:

The solution consists of two MapReduce jobs executed in sequence via mrjob's multi-step mechanism.

The pipeline is illustrated below:

MapReduce Pipeline Data Flow

Figure 1: MapReduce Pipline Data Flow – Designed in Draw.io

Key Value Design

1. Key Value Design for the Job 1 - Counting Step

Stage	Input (key → value)	Intermediate (key → value)	Output (key → value)
Job 1 **Mapper**	Offset → JSON line one record per line from HDFS split	"TOTAL"→ 1 "CAT\t{cat}" →1 "TERM\t{t}"→1 "CATTERM\t{c}\t{t}"→1 per unique term in doc (set dedup)	"TOTAL" → count "CAT\t{cat}" → count "TERM\t{t}" → count "CATTERM\t{c}\t{t}" → count grouped by key after shuffle
Job 1 **Combiner**	"TAG\t..." → [1, 1, …] local mapper output	same key) → partial sum reduces shuffle I/O sum() is associative	(same key) → partial sum forwarded to shuffle
Job 1 Reducer	"TAG\t..." → [counts] all values for one key 160 parallel tasks	aggregated count = sum(values)	None → JSON({key, count}) RawValueProtocol piped to Job 2 input

2. Key Value Design for the Job 2 — Chi-Square Computing & Ranking

Stage	**Input (key → value)**	**Intermediate (key → value)**	**Output (key → value)**
Job 2 Mapper	None → JSON line Job 1 reducer output	(pass-through) yield None, line routes all data to single reducer	None → JSON line key=None forces reduces=1
Job 2 Reducer	None → [all JSON lines] all stats in memory 1 task total	rebuild 4 counters compute $\chi^2(t,c)$ sort desc · top-75 $N\cdot(AD-BC)^2$ / $(A+C)(B+D)(A+B)(C+D)$	None → "{cat} t1:χ^2 t2:χ^2 …" None → "term1 term2 …" per-category ranked line + merged vocabulary line

Job 1 - Counting Step

Mapper

The mapper reads one JSON line at a time, extracts categories, preprocesses the combined text, and emits four types of tagged key-value pairs:

Code:

```
def mapper(self, _, line: str):
    record = json.loads(line)
    category = record.get('category')
    processed = preprocess_record(record, self._stopwords, self._min_len)
    unique_terms = {t for t in processed['clean_tokens'] if t}

    yield 'TOTAL', 1                        # total document count
    yield f'CAT\t{category}', 1             # per-category doc count
    for term in unique_terms:
          yield f'TERM\t{term}', 1          # per-term doc count (N_t)
          yield f'CATTERM\t{category}\t{term}', 1  # per (cat,term) count (A =
N_tc)
```

Using unique_terms (a set, not a list) ensures we count document frequency — each (term, category) pair is counted once per document regardless of how many times the term appears. This is the correct quantity for chi-square, which operates on binary occurrence.

Combiner

The mrjob framework re-uses the reducer logic as a combiner. Since all four emitted tag types are simple count accumulators, the combiner sums partial counts locally on each mapper node before the shuffle phase:

```
def reducer(self, key: str, values: Iterable[int]) -> Iterable[tuple]:
    count = sum(values)        # sum partial counts (works as combiner too)
    yield None, json.dumps({'key': key, 'count': count})
```

The combiner is critical for performance: the CATTERM keys alone can number in the billions across a 56 GB dataset. Local aggregation reduces network shuffle volume substantially, directly lowering Job 1 runtime.

Reducer

The reducer aggregates all counts for a given tagged key across all mapper outputs and emits a single JSON record per key. With 160 reducers configured (matching cluster resources), this stage is highly parallel. The output is a compact JSON stream: one line per distinct tag, storing the global count needed by Job 2.

Job 2 - Chi-Square Computing and Ranking

Mapper (Pass through)

The Job 2 mapper is a simple pass-through that collects all JSON lines and routes them to a single reducer (configured via mapreduce.job.reduces: 1). This is necessary because computing $\chi^2(t, c)$ requires global statistics (N, N_t, N_c) that span all categories and terms simultaneously.

Reducer (Chi-Square Computation)

The single reducer first reads all JSON lines emitted by Job 1, populating four data structures:

```
self._total_docs        # N  — total documents across all categories
self._category_docs  # N_c — doc count per category
self._term_docs         # N_t — doc count per term (across all categories)
self._cat_term_docs  # N_tc — doc count per (category, term) pair
```

It then computes χ^2 for every (term, category) pair using the formula. The four contingency values are derived as:

A = N_tc (self._cat_term_docs[category][term])

B = N_c - A (cat_docs - a)

C = N_t - A (self._term_docs[term] - a)

D = N - (A + B + C) (self._total_docs - a - b - c)

The chi-square value is then:

chi2 = N * (a*d - b*c)^2 / ((a+c) * (b+d) * (a+b) * (c+d))

```
for term, a in self._cat_term_docs[category].items():
```

```
    b = cat_docs - a
    c = self._term_docs[term] - a
    d = self._total_docs - (a + b + c)
    denom = (a + c) * (b + d) * (a + b) * (c + d)
    if denom == 0:
          chi2 = 0.0
    else:
          chi2 = self._total_docs * (a * d - b * c) ** 2 / denom
    rows.append((chi2, term))
```

The denom == 0 guard handles edge cases where a term appears in every document of every category (making it non-discriminative), or where a category has no documents.

Top-75 Selection and Merged Dictionary

```
rows.sort(key=lambda item: item[0], reverse=True)
top_items = rows[:self._top_k]       # top 75 per category

# Emit: '<category> term1:χ² term2:χ² ...'
parts = [category]
for chi2, term in top_items:
   merged_terms.add(term)
    parts.append(f'{term}:{chi2:.6f}')
yield None, ' '.join(parts)

# Final merged alphabetical dictionary
merged_line = ' '.join(sorted(merged_terms))
yield None, merged_line
```

1. Categories are iterated in sorted order, so the output file has categories in alphabetical order as required.
2. The merged dictionary line is produced last by sorting the union of all selected terms alphabetically.

Effeciency Considerations

The following design decisions were made to achieve sub-15-minute runtimes on the 56 GB dataset:

- **Combiner reuse**: The reducer function doubles as a combiner since summation is associative and commutative. This reduces shuffle volume dramatically for high-frequency terms.
- **160 reducers in Job 1**: Tuned to 27 nodes × 16 cores × ~0.37, providing high parallelism for the counting step.

- **Single reducer in Job 2**: Global chi-square computation requires all counts in one place. This is acceptable because the aggregated data (counts, not raw records) is orders of magnitude smaller than the raw input.
- **Snappy compression on map output:** Enabled via mapreduce.map.output.compress.codec = SnappyCodec, reducing network I/O during shuffle.
- **64 MB split size:** Creates ~864 parallel mappers for the 56 GB input, maximizing data locality on 12 nodes.
- **Speculative execution disabled:** Prevents wasted duplicate tasks on this workload where all tasks are expected to complete within normal variance.
- **Sort buffer tuning:** mapreduce.task.io.sort.mb = 512 and spill percent 0.90 reduce disk spills during the sort phase.
- **Binary document occurrence:** Using a set of unique tokens per document (not a list) eliminates duplicate (term, category) pairs before they are emitted, reducing both mapper output volume and downstream aggregation work.

Script running

The Code runed through the BASH File that supports three modes: **local** (for development), **demo-hadoop** (devset on cluster), and **full-hadoop** (full 56 GB dataset):

```
#!/bin/bash
# run_pipeline.sh — Execute the chi-square MapReduce pipeline
MODE=$1
if [[ -z "$MODE" ]]; then
   read -p "Select mode: 1=local, 2=demo-hadoop, 3=full-hadoop [default:1]: "
MODE
   MODE=${MODE:-1}
fi

if [[ "$MODE" == "2" || "$MODE" == "demo-hadoop" ]]; then
   echo "Running on Hadoop (Demo devset)..."
   python main.py -c pipeline_mrjob.conf -r hadoop \
        hdfs:///user/e11736088/Ex1/review_devset.json \
          --stopwords hdfs:///user/e11736088/Ex1/stopwords.txt \
          > output_pipeline_demo.txt

elif [[ "$MODE" == "3" || "$MODE" == "full-hadoop" ]]; then
   echo "Running on Hadoop (Full dataset)..."
   python main.py -c pipeline_mrjob.conf -r hadoop \
        hdfs:///dic_shared/amazon-reviews/full/reviewscombined.json \
          --stopwords hdfs:///user/e11736088/Ex1/stopwords.txt \
          > output_pipeline.txt

else
   echo "Running locally (devset)..."
   python main.py -c pipeline_mrjob.conf \
        Assets/reviews_devset.json \
```

```
          --stopwords Assets/stopwords.txt \
          > output.txt
fi
```

Hadoop Configuration (pipeline_mrjob.conf)

The mrjob configuration file sets Hadoop job parameters for optimal cluster performance:

```
runners:
  hadoop:
    hadoop_streaming_jar: /usr/lib/hadoop/tools/lib/hadoop-streaming-3.3.6.jar
    jobconf:
    # Disable speculative execution — no wasted duplicate tasks
       mapreduce.map.speculative: false
       mapreduce.reduce.speculative: false

    # Compress map output with Snappy (fast, good ratio)
       mapreduce.map.output.compress: true
       mapreduce.map.output.compress.codec:
org.apache.hadoop.io.compress.SnappyCodec

    # Sort buffer: 512 MB, spill at 90% to reduce disk writes
       mapreduce.task.io.sort.mb: 512
       mapreduce.map.sort.spill.percent: '0.90'

    # Shuffle: 27 parallel copies, 70% buffer, 66% merge threshold
       mapreduce.reduce.shuffle.parallelcopies: 27
       mapreduce.reduce.shuffle.input.buffer.percent: '0.70'
       mapreduce.reduce.shuffle.merge.percent: '0.66'

    # 64 MB splits → ~864 parallel mappers for 56 GB input
       mapreduce.input.fileinputformat.split.maxsize: 67108864

    # 160 reducers for Job 1 (27 nodes x 16 cores x ~0.37)
    # Job 2 overrides this to 1 via its own jobconf in steps()
    mapreduce.job.reduces: 160
```

Conclusions

This work implemented a scaleable two-step MapReduce pipeline for chi-square term selection on a 56 GB dataset producing ranked terms list and the merged dictionary. When performing a chi-square computation via efficiency, combiners,parallel counting(160 reducers) and the single reducer is used with the Hadoop tuning and modular design enables the strong performance and easy extensions like Distributed-Cache Optimizations.